

PyMongo Fundamentals

Description of MongoDB

MongoDB is a popular NoSQL database that stores data in flexible, JSON-like documents. Unlike traditional relational databases, MongoDB doesn't require a predefined schema, making it ideal for applications with evolving data structures. It offers horizontal scalability, high availability, and robust performance, making it suitable for various applications, from small projects to enterprise-level solutions.

Description of PyMongo

PyMongo is the official Python driver for MongoDB. It provides a straightforward way to interact with MongoDB databases from Python applications. PyMongo translates Python data types to MongoDB BSON (Binary JSON) format and vice versa, allowing developers to work with data in a Pythonic way while leveraging MongoDB's powerful features.

Setup and Installation of PyMongo

To get started with PyMongo, you need to install the library using pip:

```
pip install pymongo
```

Connecting to MongoDB via PyMongo

```
import os
from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()

# Get MongoDB connection details
mongo_host = os.getenv("MONGO_HOST")
mongo_username = os.getenv("MONGO_USERNAME")
mongo_password = os.getenv("MONGO_PASSWORD")
mongo_db = os.getenv("MONGO_DATABASE")

# Create connection URI
mongo_uri = f"mongodb+srv://{mongo_username}:{mongo_password}@{mongo_host}/{mongo_db}?r

# Create a MongoDB client
client = MongoClient(mongo_uri)
```

Accessing Assets via PyMongo

```
# Access a database
db = client.mydatabase # OR client["mydatabase"]

# Access a collection within that database
collection = db.mycollection # OR db["mycollection"]
```

Inserting Objects into PyMongo Collections

```
# Insert a single document
user = {
    "name": "John Doe",
    "email": "john.doe@example.com",
    "age": 30,
    "address": {
        "street": "123 Main St",
        "city": "New York",
        "zip": "10001"
    }
}
result = collection.insert_one(user)
print(f"Inserted document ID: {result.inserted_id}")

# Insert multiple documents
users = [
    {
        "name": "Jane Smith",
        "email": "jane.smith@example.com",
        "age": 28
    },
    {
        "name": "Bob Johnson",
        "email": "bob.johnson@example.com",
        "age": 35
    }
]
result = collection.insert_many(users)
print(f"Inserted document IDs: {result.inserted_ids}")
```

Accessing Objects from PyMongo Collections

```
# Find a single document
user = collection.find_one({"name": "John Doe"})
print(user)

# Find multiple documents
users = collection.find({"age": {"$gt": 25}})
for user in users:
    print(user)

# Count documents
count = collection.count_documents({"age": {"$lt": 30}})
print(f"Number of users under 30: {count}")

# Sort results
sorted_users = collection.find().sort("age", 1) # 1 for ascending, -1 for descending
for user in sorted_users:
    print(f"{user['name']}: {user['age']}")

# Limit results
limited_users = collection.find().limit(5)
for user in limited_users:
    print(user)

# Projection (selecting specific fields)
projection = collection.find({}, {"name": 1, "email": 1, "_id": 0}) # 1 to include, 0
for user in projection:
    print(user)
```

Resources

- [PyMongo Documentation](#)
- [MongoDB Documentation](#)